

Lab 4

Lab 3 proved the rendering pipeline worked by playing back a static sprite sheet. Lab 4 tears most of that scaffolding out and replaces it with a proper object model — one where objects know how to update and render themselves, live for a fixed duration, and are spawned interactively by the player clicking the mouse.

Models

The first visible change is structural. The root-level `GameObject.cs` and `RenderableGameObject.cs` are deleted and replaced by a richer hierarchy under a new `Models/` folder. This also introduces a fourth class, `AnimatedGameObject`, and extracts `TextureData` from inside `GameRenderer` where it previously lived as a nested struct.

`Models/TextureData.cs` is the simplest piece — just width and height, pulled out so that any class in the project can reference it without depending on `GameRenderer`:

```
public struct TextureData
{
    public int Width { get; init; }
    public int Height { get; init; }
}
```

`Models/GameObject.cs` is no longer empty. It gains an `Id` property, making every game object uniquely identifiable — which becomes important once the dictionary-based object store is introduced:

```
public class GameObject
{
    public int Id { get; }

    public GameObject(int id)
    {
        Id = id;
    }
}
```

RenderableGameObject - objects that know how to load and draw themselves

The redesigned `RenderableGameObject` takes a significant step: rather than being handed a texture ID from the outside, it loads its own texture in the constructor:

```
public RenderableGameObject(string fileName, int id) : base(id)
{
    TextureId = GameRenderer.LoadTexture(fileName, out var textureData);
    TextureInformation = textureData;
}
```

```

TextureSource = new Rectangle<int>(0, 0, textureData.Width,
textureData.Height);
TextureDestination = new Rectangle<int>(0, 0, textureData.Width,
textureData.Height);
}

```

It also gains two virtual methods that define its contract with the game loop:

```

public virtual void Render(GameRenderer renderer)
{
    renderer.RenderGameObject(this);
}

public virtual bool Update(int timeSinceLastFrame)
{
    return true; // base implementation: always alive
}

```

Update returning `bool` is the key design decision here, `false` signals that the object has finished its lifecycle and should be removed. Render delegates back to the renderer, keeping SDL concerns out of the model layer.

AnimatedGameObject - time-based sprite sheet playback

`AnimatedGameObject` extends `RenderableGameObject` and replaces the frame-counter animation from Lab 3 with something time-aware. Rather than advancing one cell per game tick, it tracks elapsed milliseconds and derives the current frame from that:

```

public AnimatedGameObject(string fileName, int durationInSeconds, int id,
    int numberOfFrames, int numberOfColumns, int numberOfRows, int x, int y)
    : base(fileName, id)
{
    _timePerFrame = (durationInSeconds * 1000) / _numberOfFrames;

    // center the sprite on the spawn point
    var halfRow = _rowHeight / 2;
    var halfColumn = _columnWidth / 2;
    TextureDestination = new Rectangle<int>(x - halfColumn, y - halfRow,
        _columnWidth, _rowHeight);
    TextureSource = new Rectangle<int>(0, 0, _columnWidth, _rowHeight);
}

public override bool Update(int timeSinceLastFrame)
{
    _timeSinceAnimationStart += timeSinceLastFrame;

    if (_timeSinceAnimationStart > _durationInSeconds * 1000) return false; //
    animation complete, remove object
}

```

```

var currentFrame = _timeSinceAnimationStart / _timePerFrame;
_currentRow = currentFrame / _numberOfColumns;
_currentColumn = currentFrame % _numberOfColumns;

TextureSource = new Rectangle<int>(
    _currentColumn * _columnWidth, _currentRow * _rowHeight,
    _columnWidth, _rowHeight);

return true;
}

```

Once `_timeSinceAnimationStart` exceeds the total duration, `Update` returns false and the object removes itself from the world. This makes it a perfect fit for one-shot effects like the one created in this lab.

GameLogic - an object store with a lifecycle

The `List<GameObject>` from Lab 3 is replaced with a `Dictionary<int, GameObject>` keyed by ID, which makes removal by ID efficient. `ProcessFrame` is now empty; all per-frame work has moved into `RenderAllObjects`:

```

public void RenderAllObjects(int timeSinceLastFrame, GameRenderer renderer)
{
    List<int> itemsToRemove = new List<int>();
    foreach (var gameObject in GetRenderables())
    {
        if (gameObject.Update(timeSinceLastFrame))
        {
            gameObject.Render(renderer);
        }
        else
        {
            itemsToRemove.Add(gameObject.Id);
        }
    }

    foreach (var item in itemsToRemove)
    {
        _gameObjects.Remove(item);
    }
}

```

The two-pass approach, collect IDs to remove, then remove them, avoids modifying the dictionary while iterating over it. Objects that return `false` from `Update` are silently dropped.

Spawning a bomb is equally straightforward:

```

public void AddBomb(int x, int y)

```

```
{
    AnimatedGameObject bomb = new AnimatedGameObject("BombExploding.png", 2,
        _bombIds, 13, 13, 1, x, y);
    _gameObjects.Add(bomb.Id, bomb);
    ++_bombIds;
}
```

This creates a 2-second explosion animation with 13 frames laid out in a 13×1 sprite sheet, centered on the click position. IDs start at 100 and increment, there is no real reason other than to demonstrate that IDs don't need to be anything specific.

GameRenderer - a singleton and a partial class split

Two notable changes happen in `GameRenderer`. First, it's split into two partial class declarations in the same file. The first part holds the render loop and `RenderGameObject`; the second holds `LoadTexture`. This is largely aesthetic organisation, separating "runtime rendering" from "asset loading".

Second, and more significantly, `LoadTexture` becomes static, backed by an `_instance` singleton:

```
private static GameRenderer? _instance;

public GameRenderer(Sdl sdl, GameWindow gameWindow, GameLogic gameLogic)
{
    // ...
    _instance = this;
}

public static int LoadTexture(string fileName, out TextureData textureData)
{
    // uses _instance!._sdl, _instance!._renderer, etc.
}
```

This exists because `RenderableGameObject` calls `LoadTexture` from its constructor, and constructors can't easily receive a renderer reference without threading it through every subclass. The singleton sidesteps that plumbing. It's a temporary shortcut, though it does mean only one renderer can exist at a time for now.

Render also gains proper frame timing. It records when each frame was rendered using `DateTimeOffset` and passes the elapsed milliseconds to `RenderAllObjects`:

```
var timeSinceLastFrame = 0;
var now = DateTimeOffset.UtcNow;
if (_lastFrameRenderedAt > DateTimeOffset.MinValue)
{
    timeSinceLastFrame =
        (int)now.Subtract(_lastFrameRenderedAt).TotalMilliseconds;
}
```

```
_gameLogic.RenderAllObjects(timeSinceLastFrame, this);  
_lastFrameRenderedAt = now;
```

This is what makes AnimatedGameObject's time-based animation actually work correctly — the animation speed is now independent of frame rate.

InputLogic - spawning bombs on click

InputLogic gains a reference to GameLogic and starts tracking mouse coordinates:

```
case (uint)EventType.Mousebuttondown:  
{  
    mouseX = ev.Motion.X;  
    mouseY = ev.Motion.Y;  
    mouseButtonStates[ev.Button.Button] = 1;  
    break;  
}
```

After the event loop drains, if the primary button is held down, a bomb is spawned at the recorded position:

```
if (mouseButtonStates[(byte)MouseButton.Primary] == 1)  
{  
    _gameLogic.AddBomb(mouseX, mouseY);  
}
```

This is checked once per frame after all events are consumed, so the coordinates always reflect the position at the most recent click.

By the end of this lab, the game has its first interactive mechanic: clicking anywhere on screen spawns an explosion animation that plays out over two seconds and then disappears.